

Data Challenge Report

Keerthana S (ED22B080)
Kaggle ID: `keerthana_pacific`
Name in the contest : **Team_Name**

November 21, 2025

1 Introduction and Dataset Exploration

amsmath

The aim of this project was to build a model that can predict the quality score of a model-generated response, given the corresponding system prompt, user prompt, and the response itself. The dataset came in the form of several JSON and NumPy files, each representing a different component of the evaluation pipeline. I began by loading and inspecting all the files inside the provided directory. This included:

- `train_data.json` and `test_data.json`, containing the actual interaction samples,
- `metric_name_embeddings.npy`, a matrix of fixed-length embeddings already embedded using Gemma.
- `metric_names.json`, which listed the names corresponding to each metric embedding.

Once loaded, I converted the JSON structures into Pandas dataframes and verified their shape. One of the first things I did was inspect the distribution of these scores. The histogram turned out to be highly skewed towards the upper end (particularly scores around 9 and 10), which suggested an imbalance that models would inevitably have to contend with. I also checked for missing values within the text fields and replaced any `NaN` values with empty strings so that the embedding model would not break.

To understand how the embeddings fit into the pipeline, I looked closely at the `embedding_idx` column in each row. This index acts as a pointer to one of the precomputed metric embeddings. These embeddings are fixed-length numerical vectors (768-dimensional in this case), loaded from the NumPy file.

For the text embeddings, I constructed a combined string for each row by joining the system prompt, user prompt, and the model response using `[SEP]` separators. I experimented with a few embedding backbones, such as:

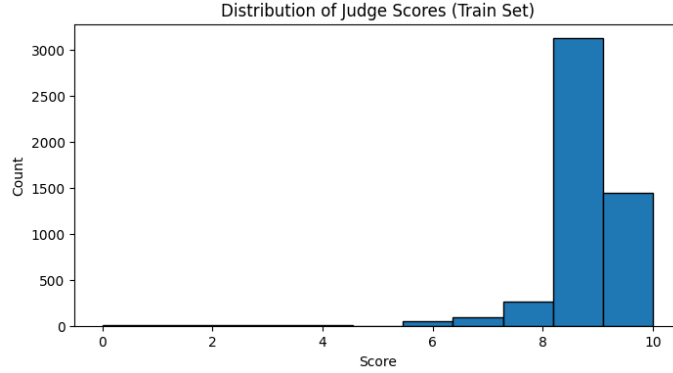


Figure 1: Distribution of scores in the training dataset.

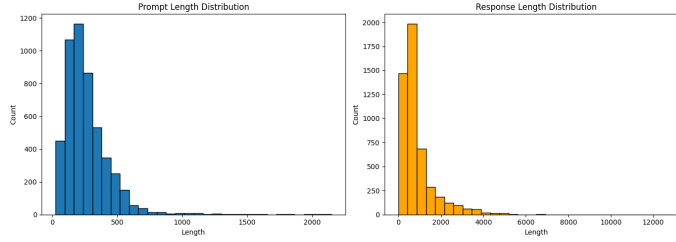


Figure 2: Distribution of prompt lengths in the training dataset.

- **Gemma-text model** (the original suggestion in the challenge),
- **SentenceTransformer models** such as `all-MiniLM-L6-v2`,
- **MPNet-based transformers**, and
- Lighter variants like `paraphrase-MiniLM-L3`.

Each embedding—both the metric embedding and the text embedding—produced a 768-dimensional vector (depending on the encoder chosen), giving a total of **1536** base features before any additional processing.

To enrich this representation, I data- engineered a set of heuristic features based on the relationship between the metric embedding and the corresponding text embedding. These included:

- Euclidean distance,
- Manhattan distance,
- Minkowski distance with $p = 3$,
- Chebyshev distance,

- Cosine similarity,
- Dot product,
- Pearson correlation,
- Norm (magnitude) of each embedding,
- Angle between the embedding vectors.

The intuition behind these heuristics was simple: distance or similarity measures may capture structural relations between the prompt-response pair and the metric embedding in a way that raw embeddings alone do not. After computing all these metrics, I concatenated them along with the original embeddings into one large feature matrix. An issue I later faced was the fact that some of these features were correlated and hence added little to no value. This became the final input for all downstream regression models.

Overall, the data exploration stage helped build intuition for how the embeddings were organized, why the score distribution might affect performance, and what additional features could strengthen the learning process beyond the plain vectors themselves.

2 Modeling Approach 1

With the preprocessing complete, I moved on to the modeling phase. The broad idea was to evaluate how well classical regression models could perform on top of the enriched feature matrix. Since the embeddings already capture expressive semantic information, I started with relatively simple models to see how far they could go.

2.1 Linear Regression

I first trained a Linear Regression model on a standardized version of the feature matrix.

The Linear Regression model minimizes the classical squared error:

$$\min_{\mathbf{w}} \frac{1}{N} \sum_{i=1}^N (y_i - \mathbf{w}^\top \mathbf{x}_i)^2.$$

The model was evaluated using 5-fold cross-validation with RMSE as the metric. The model performed pretty poorly, as I assumed it was more complex than a Linear Relationship and was also very prone to over-fitting. The RMSE on the test dataset was around 4.6 in this case showing that the model performed pretty badly on it.

2.2 Ridge Regression

To combat the overfitting observed with plain linear regression, I next tried Ridge Regression. This model adds an ℓ_2 penalty to the weights, which stabilizes the training process in high-dimensional spaces. I experimented with different values of the regularization parameter α , including 1, 10, 50, and 100. The value $\alpha = 10$ consistently gave the best trade-off between bias and variance. The key parameters for my final Ridge model were:

- **alpha**: 10.0,
- **solver**: auto,
- training on scaled features,
- evaluation using 5-fold cross-validation.

Ridge outperformed the plain linear model, suggesting that the heuristic features and embeddings together introduce some correlations whose weights can be shrunk.

Ridge regression minimizes the regularized loss:

$$\min_{\mathbf{w}} \frac{1}{N} \sum_{i=1}^N (y_i - \mathbf{w}^\top \mathbf{x}_i)^2 + \alpha \mathbf{w}_2^2.$$

2.3 LightGBM Regressor

Parallel to experimenting with linear models, I also wanted to test a stronger non-linear baseline. For this, I trained a LightGBM regressor, since tree-based algorithms are well known for handling heterogeneous features without requiring scaling. I therefore used the unscaled version of the full feature matrix. The configuration of the model I used post tuning using a grid search was:

- **n_estimators**: 500,
- **learning_rate**: 0.05,
- **num_leaves**: 31,
- **subsample**: 0.8,
- **colsample_bytree**: 0.8,
- evaluation using 5-fold cross-validation.

LightGBM consistently outperformed both linear regression and Ridge regression. This result made sense because tree-based models are particularly good at splitting on non-linear heuristic features such as cosine similarity, Manhattan distance, and Euclidean distance—features that a linear model cannot

leverage as effectively. The LightGBM model is optimized using gradient boosting over decision trees, where at each stage the model fits a tree to the negative gradients (pseudo-residuals). The objective minimized is:

$$\mathcal{L} = \sum_{i=1}^N \ell(y_i, \hat{y}_i) + \sum_{t=1}^T \Omega(f_t),$$

where ℓ is typically squared loss,

$$\ell(y_i, \hat{y}_i) = (y_i - \hat{y}_i)^2,$$

and $\Omega(f_t)$ penalizes tree complexity (depth, leaves, etc.). However, at this point I also took a closer look at the score distribution and noticed how strongly the dataset was skewed towards the values 9 and 10. The model therefore tended to overpredict these high scores, simply because the training process saw far more examples from the upper range. To compensate for this imbalance, I introduced inverse-frequency sample weights during training. The idea was simple: samples belonging to underrepresented scores (for example, 3, 4, 5) should receive higher weight, and those belonging to the majority classes (scores 9 and 10) should receive less weight.

```
value_counts = train_df['score'].value_counts().to_dict()
sample_weight = train_df['score'].map(lambda x: 1.0 / value_counts[x]).values
sample_weight = sample_weight / sample_weight.mean()
```

This weighting strategy ensured that the model no longer treated the skewed score distribution as a signal to favour high outputs. Although this adjustment improved cross-validation stability, the RMSE on the held-out test set still hovered around 3.5. This indicated that while the model was capturing some meaningful structure, there remained significant room for improvement, possibly through deeper architectures.

3 Modeling Approach 2: Contrastive Learning, Negative Sampling, and MLP Regression

After completing the classical-model exploration, I wanted to try a more representation-driven approach that could directly learn the relationship between a text embedding and its corresponding metric embedding. This led me towards contrastive learning, a framework in which the model is encouraged to assign higher similarity scores to correct (text, metric) pairs and lower scores to mismatched or “negative” pairs. The basis of this idea stemmed from the simple negative sampling idea- the fact that there are lesser “bad” samples for the model to learn could push us to artificially generate these bad samples and augment the dataset accordingly!

3.1 Understanding Negative Sampling in This Context

Before implementing the model, I spent some time understanding the role of negative sampling in contrastive setups. In the context of this task, each training sample has one correct metric embedding—the metric corresponding to its row in the dataset. However, the model has no explicit reason to know that this metric should be considered “positive” unless we expose it to examples of what *incorrect* metrics look like.

For every (text, metric) pair, I generated several “wrong” metric embeddings by randomly selecting metric names belonging to other samples. During training, the model was encouraged to give high scores for the correct pair and low scores for the randomly sampled negatives. This created a simple but effective training signal that does not require explicit labels.

Formally, for each text embedding t , we form:

- one **positive** metric embedding m^+ , and
- k **negative** metric embeddings $\{m_1^-, m_2^-, \dots, m_k^-\}$,

and train the model so that $f(t, m^+)$ is ranked higher than $f(t, m^-)$ for all negatives.

This structure would later form the basis for training not only using a contrastive loss but also with ranking-style losses such as InfoNCE.

3.2 Model Architecture: A Simple MLP on Concatenated Embeddings

For the model architecture, I intentionally kept the design minimal to see how the basic structure performs. Each input consists of a 768-dimensional text embedding concatenated with a 768-dimensional metric embedding, giving a 1536-dimensional combined vector. I did not add any type of extra features as I wanted to use an MLP that will anyways learn those extra features by itself. I used a small multi-layer perceptron (MLP) of the form:

```
Linear(1536 → 512)
ReLU (or GELU)
Linear(512 → 1)
```

The output of the network is a single scalar logit representing the compatibility between the given text and metric embeddings. This raw logit is passed through a sigmoid and scaled to the range 0–10 to match the score distribution.

I later added **LayerNorm** before the first linear layer. This was helpful in stabilizing training because the magnitudes of the concatenated embeddings varied significantly across samples. LayerNorm ensured that the model did not become overly sensitive to the scale of particular dimensions and made the training loss much more consistent across epochs.

3.3 Exploring Different Loss Functions

After building the architecture, I experimented with several learning objectives, each capturing a slightly different kind of supervision:

- **Contrastive Loss:**

$$\mathcal{L} = -\log \sigma(f(t, m^+)) - \log (1 - \sigma(f(t, m^-)))$$

This loss encourages the model to strongly separate positive and negative pairs. It worked well as an unsupervised pre-training step before regression.

- **Regression Loss (MSE):** After contrastive pre-training, I trained the same model to directly predict the numeric score using mean squared error. Applying a sigmoid and scaling by 10 aligned the outputs with the valid score range.
- **Ranking Loss / InfoNCE Loss:** To improve the model’s discriminative ability, I also experimented with InfoNCE:

$$\mathcal{L}_{NCE} = -\log \frac{\exp(f(t, m^+))}{\exp(f(t, m^+)) + \sum_{j=1}^k \exp(f(t, m_j^-))}$$

This loss proved more stable than the simple contrastive objective and significantly improved validation scores.

Interestingly, combining the contrastive (or InfoNCE) phase with a regression fine-tuning phase yielded much better performance than using either loss alone. The contrastive phase gave the model a good sense of the metric space, while regression helped it align predictions to the actual numerical scale.

3.4 Cross-Validation, Score Discretization, and Rounding Experiments

To ensure robustness, I used 5-fold stratified cross-validation. Since the target scores are continuous, I discretized them into quantile bins using: `KBinsDiscretizer(n_bins=10, strategy="quantile")` for stratification. This ensured that each fold had comparable representation of low, medium, and high scores.

After predictions were produced, I experimented with:

- **rounding** the predicted scores,
- **flooring** the scores,
- and **leaving them continuous**.

Since the true labels are integers, I wanted to see whether discretizing the outputs would reduce error. Surprisingly, the continuous predictions consistently performed best, likely because rounding introduced unnecessary bias near score boundaries. The best RMSE from this shot up to nearly 3.2 from 3.6, validating the hypothesis that I had about using deeper architectures.

3.5 Exploring Dual Encoders, Expert Heads, and Shared Embedding Spaces

After obtaining encouraging results from the contrastive+regression pipeline, I explored a more expressive class of models built around dual encoders. The central idea was to learn two separate projection functions:

- one that transforms the **text embedding** into a latent representation, and
- another that transforms the **metric embedding** into the same shared space.

The hope was that if both embeddings were projected into a common latent space, the model would have a cleaner, geometry-driven way of deciding whether a prompt-response pair matches a given metric. This was inspired by sentence similarity models where two encoders are trained to produce comparable embeddings for different modalities. The best part about this approach was the fact that I could use the metrics embedded by one approach and the response-pairs in another as this basically allows dual encoding learning.

Formally, I used: $h_t = f_\theta(t)$, $h_m = g_\phi(m)$, where f_θ and g_ϕ are two small feed-forward networks. Their outputs were normalized and compared using dot-product similarity:

$$s = \langle h_t, h_m \rangle.$$

This similarity score was then fed into one or more regression heads. I tested dual encoder+ two regression heads initially.

3.5.1 Dual Regression Heads: Low-Score Expert and High-Score Expert

To push the idea further, I experimented with a two-expert setup. The motivation was that low-scoring samples (scores below 5) exhibit very different patterns from high-scoring ones. For example, low scores often indicate factual hallucination or irrelevance, whereas high scores correspond to fluency and correctness. To capture these differences, I built a pair of regression heads:

- **Low-score expert** for samples where similarity was below a threshold.
- **High-score expert** for samples where similarity was above a threshold.

Routing was performed based on the similarity score obtained from the dual encoders. The idea was that the dual encoder first determines how well the text and metric align, and then the appropriate expert head refines the numeric prediction.

This architecture looked as follows:

```
Text Embedding -----> Text Projector ----\
                                           > Similarity --> Expert-Low
Metric Embedding ---> Metric Projector --/      --> Expert-High
```

3.5.2 Loss Design: InfoNCE, Ranking Loss, and Regression

The training procedure used a combination of:

- **InfoNCE loss**, which treats each (text, metric) pair as a positive example and k randomly sampled metric embeddings as negatives,
- **Ranking loss**, penalizing situations where a negative metric is scored higher than a positive one,
- **Regression loss** on the predictions from the expert heads.

The typical pattern for a batch was:

1. Compute similarity to the correct metric.
2. Sample 2–3 incorrect metrics (negative sampling).
3. Apply InfoNCE or ranking loss on similarities.
4. Pass the sample to either the low or high expert for score regression.

This multi-objective learning goal forced the model to learn:

- how to discriminate correct vs. incorrect metrics,
- how to place low-score and high-score samples in different regions of the space, and
- how to regress the final scalar score.

This architecture did slightly better than the simpler models. The RMSE scores hovered around 2.3–2.4, sometimes worse depending on hyperparameters. After several trials, I identified possible reasons why the score ceased to improve beyond tuning/more training:

- **Data size**: the dataset is relatively small, and dual encoders double the number of learnable parameters. This likely caused overfitting.
- **Routing instability**: deciding whether a sample should go to the low-score or high-score head based on a learned similarity score sometimes introduced noise, especially early in training.
- **Multi-task conflict**: optimizing contrastive, ranking, and regression losses simultaneously caused the gradients to compete, and there was not enough data to stabilize the learning dynamics.

3.5.3 Modeling Approach 3: Simpler MLP or Dual Encoder with One Regression Head

As the previous setup was a little more complicated than expected, I decided to use a Dual Encoder with a single Regressor head. I tried both iterations, one of using the precomputed embeddings+ MPNet embeddings for the response+pair and one with both encoded with MPNet. The second one performed better, though the first approach also had a very close score. Encoding both with Gemma also gave similar results. After many experiments, the configuration that performed best was ironically the simplest:

- **Dual Encoder** operating on the concatenated (text, metric) embeddings,
- **One regression head**,
- **Gentle LayerNorm** on the input space,
- **Contrastive pretraining + regression fine-tuning**,
- **Tuned learning rate and batch size**.

3.6 Final Model Architecture and Performance

After experimenting with several architectures, the best-performing model was built around a lightweight dual-encoder framework with a single regression head. The core of this design was the **TwoHeadProjector** module, which independently projected the text embedding and the metric embedding into a shared latent space. Each branch first applied a lightweight, learnable normalization layer (**GentleLN**), which behaves like a “soft” LayerNorm: it normalizes only with respect to the mean and variance of each vector, but with learned scaling parameters and without the rigid constraints of standard LayerNorm. This choice preserved the semantic geometry of the original SBERT embeddings while still reducing instability during contrastive training.

$$\text{GentleLN}(z) = \frac{z - \mu(z)}{\sqrt{\sigma^2(z) + \varepsilon}} \odot w + b,$$

where w and b are learned scale and bias parameters. This “soft” normalization stabilized training without destroying the geometry of the SBERT embedding space.

Each normalized embedding was passed through a small feed-forward network:

$$f_\theta(x) = \text{FFN}(x), \quad g_\phi(m) = \text{FFN}(m),$$

followed by L2 normalization:

$$\hat{x} = \frac{f_\theta(x)}{\|f_\theta(x)\|_2}, \quad \hat{m} = \frac{g_\phi(m)}{\|g_\phi(m)\|_2}.$$

Their interaction was computed using a scaled dot-product:

$$s = \exp(t_{log}) \cdot \langle \hat{x}, \hat{m} \rangle,$$

where t_{log} is a **learnable temperature parameter**, allowing the model to adjust how sharply it separates positive vs. negative metric-text pairs.

On top of this similarity score, a regression head predicted a continuous output:

$$\hat{y} = h_{\psi}(s),$$

and the final prediction was obtained by:

$$score = 10 \cdot \sigma(\hat{y}),$$

ensuring all outputs lie in the 0–10 range.

The overall training objective combined multiple components:

1. Regression loss (MSE):

$$\mathcal{L}_{reg} = (\sigma(\hat{y}) - y)^2.$$

2. Contrastive loss for positive pairs:

$$\mathcal{L}_{pos} = \text{BCEWithLogits}(s_{pos}, y),$$

3. Contrastive loss for negative (random) metric samples:

$$\mathcal{L}_{neg} = \text{BCEWithLogits}(s_{neg}, 0),$$

4. Margin-based penalty:

$$\mathcal{L}_{margin} = \max(0, M - (s_{pos} - E_k[s_{neg,k}])),$$

where M is a margin hyperparameter.

The final loss used during training was:

$$\mathcal{L} = \lambda_{reg}\mathcal{L}_{reg} + w_{pos}\mathcal{L}_{pos} + w_{neg}\mathcal{L}_{neg} + w_{mrg}\mathcal{L}_{margin}.$$

The final output was obtained by applying a sigmoid to the regression score and rescaling it back to the 0–10 range. Training for **20–30 epochs** consistently produced the best results, with the optimal performance around **25 epochs**; beyond 40 epochs the model began to overfit, as indicated by improving contrastive loss but worsening validation RMSE.

Thanks to its compact architecture, gentle normalization, and balanced combination of contrastive and regression signals, this model achieved test RMSE scores in the range:

$$\mathbf{1.7\text{--}1.8}.$$

outperforming all other approaches explored earlier. Interestingly, the simpler MLP variant with a single regression head also performed competitively with an RMSE of around 1.9, but the dual encoder with soft normalization and temperature scaling consistently delivered the strongest scores while remaining relatively easy to train. We can also see the graph below that the middle and lower values are also getting predicted, something that was a problem with the simple linear regression models!

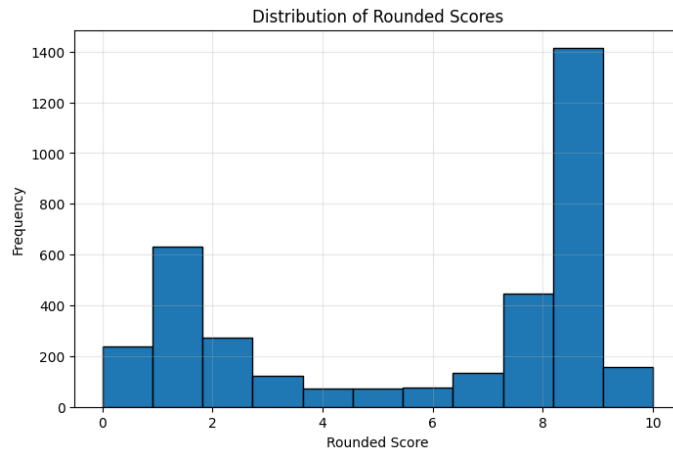


Figure 3: Final distribution of the scores predicted.